

The second part of this article teaches readers how to use the OpenCV library via the Python interface to perform image processing-based tasks. The previous article gave a brief overview of embedded vision and the various components required to make it work. It also covered the installation procedure for the OpenCV library.



As I had mentioned in the previous article, OpenCV is a very powerful library that is used for image processing. I will now explain its syntax and how to use it, along with a few examples. I will not go into the basics of Python code and syntax, since this is extensively documented; see <http://www.python.org/doc/> for more.



Note: The source code used in this article can be downloaded from http://linuxforu.com/article_source_code/feb12/image_processing.zip.

Reading and displaying an image

Our first example, which I have named 'Test.py' is short:

```
#!/usr/bin/python
import cv
```

```
src = cv.LoadImage("/home/jayneil/12.png")
cv.NamedWindow("RGB", cv.CV_WINDOW_AUTOSIZE)
cv.ShowImage("RGB", src)
cv.WaitKey(500)
cv.DestroyWindow("RGB")
```

In a terminal, navigate to the directory that contains the file, and run it with 'python Test.py'. The output can be seen in Figure 1—our image in a window. Import the OpenCV module 'cv' so that you can use it, then call the *LoadImage* function (passing the full path to the file, which is needed) to load the image into the variable *src*. The *NamedWindow* function creates a window in which to display the loaded image. The first parameter is the name for the window, and the second sets the sizing mode to automatically resize the window to the size of the image. Next, *ShowImage* is used to display the image in